In [ ]:

```python
# Setup: import required libraries and define local appendix paths.
from pathlib import Path
import re
import warnings

import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency, spearmanr, kendalltau
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore", category=FutureWarning)

DATA_PATH = Path("data/data_cleaned.csv")
VARIABLE_DICTIONARY_PATH = Path("data/variable_dictionary.csv")
OUTPUT_DIR = Path("outputs")
TABLE_DIR = OUTPUT_DIR / "descriptive_tables"
PLOT_DIR = OUTPUT_DIR / "plots"

for directory in [OUTPUT_DIR, TABLE_DIR, PLOT_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    "figure.figsize": (9, 5),
    "figure.dpi": 140,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
})
```

In [2]:

```python
# Load prepared Phase 1 survey data; cleaning and renaming were completed before appendix analysis.
df = pd.read_csv(DATA_PATH, sep=";", encoding="utf-8-sig")
variable_dictionary = pd.read_csv(VARIABLE_DICTIONARY_PATH, encoding="utf-8-sig")

print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
print(df.columns.tolist())
df.head()
```

```
Rows: 187
Columns: 28
['participant', 'q01_formal_structure', 'q02_lessons_learned', 'q03_solution_source', 'q04_knowledge_base', 'q05_expert_finder', 'q06_project_knowledge', 'q07_hidden_knowledge_exists', 'q08_knowledge_utilization', 'q09_hidden_knowledge_value', 'q10_fear_of_sharing', 'q11_share_with_colleagues', 'q12_share_with_man
agement', 'q13_personality_similarity', 'q14_extrovert_self_assessment', 'q15_prior_personality_test', 'q16_personality_test_willingness', 'q17_profile_use_consent', 'q18_ai_helpfulness', 'q19_ai_internal_channel_analysis', 'q20_ai_public_profile_analysis', 'q21_ai_work_content_analysis', 'q22_gender', 'q23_age_gr
oup', 'q24_organization_size', 'q25_tenure', 'q26_education', 'q27_decision_level']
```

Out[2]:

	participant	q01_formal_structure	q02_lessons_learned	q03_solution_source	q04_knowledge_base	q05_expert_finder	q06_project_knowledge	q07_hidden_knowledge_exists	q08_knowledge_utilization	q09_hidden_knowledge_value	...	q18_ai_helpfulness	q19_ai_internal_channel_analysis	q20_ai_public_profile_analysis	q21_ai_work_content_analysis
0	1	To some level	Yes	Official internal collaboration channels (emai...	Yes	Periodically	No	Yes	25% to 50%	Large	...	Up to a certain level Yes	With certain conditions Yes	With certain conditions Yes	Yes
1	2	To some level	Periodically	Official internal collaboration channels (emai...	We don't have such a system	We don't have such a system	We don't have such a system	Yes	Completely unutilized	Large	...	Up to a certain level Yes	With certain conditions Yes	No	No
2	3	Yes	Yes	Official internal collaboration channels (emai...	Yes	Periodically	Yes	Yes	More than 75%	Medium	...	Yes	Yes	No	Yes
3	4	To some level	Periodically	Official internal collaboration channels (emai...	No	We don't have such a system	No	Yes	25% to 50%	Large	...	Yes	With certain conditions Yes	No	Yes
4	5	To some level	Periodically	Official internal collaboration channels (emai...	Periodically	We don't have such a system	We don't have such a system	Yes	50% to 75%	Large	...	Up to a certain level Yes	No	No	With certain conditions Yes

5 rows × 28 columns

In [3]:

```python
# Validate the prepared dataset schema and preserve all valid responses.
EXPECTED_COLUMNS = ["participant"] + [f"q{i:02d}_" for i in range(1, 28)]
missing_required_prefixes = [prefix for prefix in EXPECTED_COLUMNS if not any(column.startswith(prefix) for column in df.columns)]

if missing_required_prefixes:
    raise ValueError(f"Missing expected variables or prefixes: {missing_required_prefixes}")

if df["participant"].duplicated().any():
    raise ValueError("Participant identifiers are not unique.")

schema_audit = pd.DataFrame({
    "metric": ["rows", "columns", "unique_participants", "missing_participant_ids"],
    "value": [df.shape[0], df.shape[1], df["participant"].nunique(), df["participant"].isna().sum()],
})
schema_audit.to_csv(TABLE_DIR / "prepared_dataset_schema_audit.csv", index=False)
schema_audit
```

Out[3]:

	metric	value
0	rows	187
1	columns	28
2	unique_participants	187
3	missing_participant_ids	0

In [4]:

```python
# Inspect missing values and observed categorical values in the prepared data.
def unique_values_table(dataframe):
    rows = []
    for column in dataframe.columns:
        values = dataframe[column].dropna().astype(str).map(str.strip)
        unique_values = sorted(values.unique())
        rows.append({
            "column": column,
            "unique_count": len(unique_values),
            "unique_values": " | ".join(unique_values),
        })
    return pd.DataFrame(rows)

missing_summary = df.isna().sum().rename("missing_count").to_frame()
missing_summary["missing_percent"] = (missing_summary["missing_count"] / len(df) * 100).round(2)
missing_summary.to_csv(TABLE_DIR / "missing_values.csv")

unique_values = unique_values_table(df)
unique_values.to_csv(TABLE_DIR / "unique_values_by_column.csv", index=False)

missing_summary
```

Out[4]:

	missing_count	missing_percent
participant	0	0.00
q01_formal_structure	0	0.00
q02_lessons_learned	0	0.00
q03_solution_source	0	0.00
q04_knowledge_base	0	0.00
q05_expert_finder	0	0.00
q06_project_knowledge	0	0.00
q07_hidden_knowledge_exists	0	0.00
q08_knowledge_utilization	0	0.00
q09_hidden_knowledge_value	1	0.53
q10_fear_of_sharing	0	0.00
q11_share_with_colleagues	0	0.00
q12_share_with_management	0	0.00
q13_personality_similarity	0	0.00
q14_extrovert_self_assessment	0	0.00
q15_prior_personality_test	0	0.00
q16_personality_test_willingness	0	0.00
q17_profile_use_consent	0	0.00
q18_ai_helpfulness	0	0.00
q19_ai_internal_channel_analysis	0	0.00
q20_ai_public_profile_analysis	0	0.00
q21_ai_work_content_analysis	0	0.00
q22_gender	0	0.00
q23_age_group	0	0.00
q24_organization_size	0	0.00
q25_tenure	0	0.00
q26_education	0	0.00
q27_decision_level	0	0.00

In [5]:

```python
# Create frequency and percentage tables for all survey variables.
QUESTION_COLUMNS = [column for column in df.columns if re.match(r"q\d{2}_", column)]

def frequency_table(dataframe, column):
    counts = dataframe[column].value_counts(dropna=False).rename_axis("category").reset_index(name="count")
    counts["category"] = counts["category"].astype("string").fillna("Missing")
    counts["percent"] = (counts["count"] / len(dataframe) * 100).round(2)
    return counts

combined_frequency_tables = []
for column in QUESTION_COLUMNS:
    table = frequency_table(df, column)
    table.insert(0, "variable", column)
    table.to_csv(TABLE_DIR / f"{column}_frequency.csv", index=False)
    combined_frequency_tables.append(table)

all_frequency_tables = pd.concat(combined_frequency_tables, ignore_index=True)
all_frequency_tables.to_csv(TABLE_DIR / "all_frequency_tables.csv", index=False)

all_percentage_tables = all_frequency_tables[["variable", "category", "percent"]].copy()
all_percentage_tables.to_csv(TABLE_DIR / "all_percentage_tables.csv", index=False)

all_frequency_tables.head(20)
```

Out[5]:

	variable	category	count	percent
0	q01_formal_structure	Yes	121	64.71
1	q01_formal_structure	To some level	55	29.41
2	q01_formal_structure	No	11	5.88
3	q02_lessons_learned	Periodically	92	49.20
4	q02_lessons_learned	Yes	56	29.95
5	q02_lessons_learned	No	39	20.86
6	q03_solution_source	Official internal collaboration channels (emai...	30	16.04
7	q03_solution_source	Official internal collaboration channels (emai...	28	14.97
8	q03_solution_source	Official internal collaboration channels (emai...	20	10.70
9	q03_solution_source	Official internal collaboration channels (emai...	16	8.56
10	q03_solution_source	Official internal collaboration channels (emai...	16	8.56
11	q03_solution_source	Official internal collaboration channels (emai...	9	4.81
12	q03_solution_source	Official internal collaboration channels (emai...	7	3.74
13	q03_solution_source	Other	6	3.21
14	q03_solution_source	Publicly available sources on the Internet, Ot...	6	3.21
15	q03_solution_source	Official internal collaboration channels (emai...	6	3.21
16	q03_solution_source	Publicly available sources on the Internet	5	2.67
17	q03_solution_source	Official internal collaboration channels (emai...	4	2.14
18	q03_solution_source	Internal knowledge base system	4	2.14
19	q03_solution_source	Official internal collaboration channels (emai...	4	2.14

In [6]:

```python
# Parse q03 as a multi-response item with exact handling of the standalone Other option.
Q03_OPTIONS = [
    "Official internal collaboration channels (email, slack, teams, etc.)",
    "Internal knowledge base system",
    "Publicly available sources on the Internet",
    "Public social networks",
    "Other publicly available knowledge sources",
    "Other",
]

def normalize_text(text):
    text = str(text).lower().replace("", "")
    text = re.sub(r"[^a-z0-9%]+", " ", text)
    return re.sub(r"\s+", " ", text).strip()

def option_to_column_name(option):
    return "q03_source_" + normalize_text(option).replace("%", "pct").replace(" ", "_")

def parse_q03_multiselect(value, options):
    if pd.isna(value):
        return set()
    remaining = str(value).strip()
    if not remaining:
        return set()

    selected = set()
    non_other_options = sorted([option for option in options if option != "Other"], key=len, reverse=True)
    for option in non_other_options:
        if option.lower() in remaining.lower():
            selected.add(option)
            remaining = re.sub(re.escape(option), "", remaining, flags=re.IGNORECASE)

    residual_tokens = [token.strip() for token in remaining.split(",") if token.strip()]
    if any(normalize_text(token) == "other" for token in residual_tokens):
        selected.add("Other")
    return selected

def split_multiselect(series, options):
    parsed = series.map(lambda value: parse_q03_multiselect(value, options))
    indicators = pd.DataFrame(index=series.index)
    for option in options:
        indicators[option_to_column_name(option)] = parsed.map(lambda selected: int(option in selected))
    return indicators

q03_indicators = split_multiselect(df["q03_solution_source"], Q03_OPTIONS)
q03_source_summary = q03_indicators.sum().rename("count").to_frame()
q03_source_summary["percent"] = (q03_source_summary["count"] / len(df) * 100).round(2)
q03_source_summary.to_csv(TABLE_DIR / "q03_solution_source_multiselect_summary.csv")
q03_source_summary
```

Out[6]:

	count	percent
q03_source_official_internal_collaboration_channels_email_slack_teams_etc	159	85.03
q03_source_internal_knowledge_base_system	58	31.02
q03_source_publicly_available_sources_on_the_internet	127	67.91
q03_source_public_social_networks	18	9.63
q03_source_other_publicly_available_knowledge_sources	64	34.22
q03_source_other	34	18.18

In [7]:

```python
# Generate dissertation appendix plots from the prepared categorical data.
PREFERRED_ORDER = {
    "q07_hidden_knowledge_exists": ["Yes", "No"],
    "q08_knowledge_utilization": ["Completely unutilized", "Less than 25%", "25% to 50%", "50% to 75%", "More than 75%", "100% utilized"],
    "q09_hidden_knowledge_value": ["Very low", "Low", "Medium", "Large", "Very large"],
    "q11_share_with_colleagues": ["No", "With certain compensation Yes", "Yes"],
    "q12_share_with_management": ["No", "With certain compensation Yes", "Yes"],
    "q13_personality_similarity": ["No", "It doesn't matter to me", "Yes"],
    "q16_personality_test_willingness": ["No", "With certain conditions Yes", "Yes"],
    "q18_ai_helpfulness": ["No", "Up to a certain level Yes", "Yes"],
    "q19_ai_internal_channel_analysis": ["No", "Up to a certain level Yes", "With certain conditions Yes", "Yes"],
    "q20_ai_public_profile_analysis": ["No", "Up to a certain level Yes", "With certain conditions Yes", "Yes"],
    "q21_ai_work_content_analysis": ["No", "Up to a certain level Yes", "With certain conditions Yes", "Yes"],
}

VARIABLE_LABELS = {
    "q07_hidden_knowledge_exists": "Existence of Hidden Knowledge",
    "q08_knowledge_utilization": "Estimated Internal Knowledge Utilization",
    "q09_hidden_knowledge_value": "Perceived Economic Value of Hidden Knowledge",
    "q11_share_with_colleagues": "Willingness to Share with Colleagues",
    "q12_share_with_management": "Willingness to Share with Management",
    "q13_personality_similarity": "Preference for Personality Similarity",
    "q16_personality_test_willingness": "Willingness to Participate in Personality Assessment",
    "q18_ai_helpfulness": "Perceived Helpfulness of AI",
    "q19_ai_internal_channel_analysis": "Consent for AI Analysis of Internal Communication",
    "q20_ai_public_profile_analysis": "Consent for AI Analysis of Public Profiles",
    "q21_ai_work_content_analysis": "Consent for AI Analysis of Work Content",
}

PLOT_FILES = {
    "q07_hidden_knowledge_exists": "hidden_knowledge_existence.png",
    "q08_knowledge_utilization": "internal_knowledge_utilization.png",
    "q09_hidden_knowledge_value": "hidden_knowledge_economic_value.png",
    "q11_share_with_colleagues": "share_with_colleagues.png",
    "q12_share_with_management": "share_with_management.png",
    "q13_personality_similarity": "personality_similarity_preference.png",
    "q16_personality_test_willingness": "personality_assessment_willingness.png",
    "q18_ai_helpfulness": "ai_helpfulness.png",
    "q19_ai_internal_channel_analysis": "ai_internal_channel_analysis_consent.png",
    "q20_ai_public_profile_analysis": "ai_public_profile_analysis_consent.png",
    "q21_ai_work_content_analysis": "ai_work_content_analysis_consent.png",
}

def ordered_counts(dataframe, column):
    counts = dataframe[column].value_counts(dropna=False)
    order = [category for category in PREFERRED_ORDER.get(column, []) if category in counts.index]
    remaining = [category for category in counts.index if category not in order]
    return counts.reindex(order + remaining)

def plot_bar_distribution(dataframe, column, filename):
    counts = ordered_counts(dataframe, column)
    percents = counts / counts.sum() * 100
    labels = ["Missing" if pd.isna(label) else str(label) for label in percents.index]

    fig, ax = plt.subplots(figsize=(9, 5))
    bars = ax.bar(labels, percents.values, color="#4C78A8", edgecolor="#2F3A45", linewidth=0.6)
    ax.set_title(VARIABLE_LABELS.get(column, column))
    ax.set_ylabel("Respondents (%)")
    ax.set_ylim(0, max(100, percents.max() + 10))
    ax.grid(axis="y", alpha=0.25)
    ax.set_axisbelow(True)
    ax.tick_params(axis="x", rotation=25)

    for bar, pct, count in zip(bars, percents.values, counts.values):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1,
                f"{pct:.1f}%\n(n={int(count)})", ha="center", va="bottom", fontsize=8)

    fig.tight_layout()
    fig.savefig(PLOT_DIR / filename, bbox_inches="tight")
    plt.close(fig)

for column, filename in PLOT_FILES.items():
    plot_bar_distribution(df, column, filename)

sorted(path.name for path in PLOT_DIR.glob("*.png"))
```

Out[7]:

```
['ai_helpfulness.png',
 'ai_internal_channel_analysis_consent.png',
 'ai_public_profile_analysis_consent.png',
 'ai_work_content_analysis_consent.png',
 'hidden_knowledge_economic_value.png',
 'hidden_knowledge_existence.png',
 'internal_knowledge_utilization.png',
 'personality_assessment_willingness.png',
 'personality_similarity_preference.png',
 'share_with_colleagues.png',
 'share_with_management.png']
```

In [8]:

```python
# Run original exploratory chi-square tests with sparse expected-count diagnostics.
def cramers_v(confusion_matrix):
    chi2 = chi2_contingency(confusion_matrix, correction=False)[0]
    n = confusion_matrix.to_numpy().sum()
    if n == 0:
        return np.nan
    phi2 = chi2 / n
    r, k = confusion_matrix.shape
    phi2corr = max(0, phi2 - ((k - 1) * (r - 1)) / (n - 1)) if n > 1 else 0
    rcorr = r - ((r - 1) ** 2) / (n - 1) if n > 1 else r
    kcorr = k - ((k - 1) ** 2) / (n - 1) if n > 1 else k
    denominator = min((kcorr - 1), (rcorr - 1))
    return np.sqrt(phi2corr / denominator) if denominator > 0 else np.nan

def chi_square_test(dataframe, variable_x, variable_y, analysis_type="original exploratory"):
    subset = dataframe[[variable_x, variable_y]].dropna()
    contingency = pd.crosstab(subset[variable_x], subset[variable_y])

    if contingency.shape[0] < 2 or contingency.shape[1] < 2:
        return {
            "analysis_type": analysis_type,
            "variable_x": variable_x,
            "variable_y": variable_y,
            "n": len(subset),
            "chi2": np.nan,
            "dof": np.nan,
            "p_value": np.nan,
            "bias_corrected_cramers_v": np.nan,
            "minimum_expected_count": np.nan,
            "expected_cells_below_5_percent": np.nan,
            "interpretation": "insufficient category variation",
        }

    chi2, p_value, dof, expected = chi2_contingency(contingency, correction=False)
    low_expected = int((expected < 5).sum())
    total_cells = int(expected.size)

    return {
        "analysis_type": analysis_type,
        "variable_x": variable_x,
        "variable_y": variable_y,
        "n": len(subset),
        "chi2": round(chi2, 4),
        "dof": int(dof),
        "p_value": round(p_value, 6),
        "bias_corrected_cramers_v": round(cramers_v(contingency), 4),
        "minimum_expected_count": round(float(expected.min()), 4),
        "expected_cells_below_5_percent": round(low_expected / total_cells * 100, 2),
        "interpretation": "exploratory due to sparse expected counts" if low_expected else "meets expected-count screening criterion",
    }

TEST_PAIRS = [
    ("q13_personality_similarity", "q16_personality_test_willingness"),
    ("q13_personality_similarity", "q17_profile_use_consent"),
    ("q18_ai_helpfulness", "q19_ai_internal_channel_analysis"),
    ("q18_ai_helpfulness", "q21_ai_work_content_analysis"),
    ("q24_organization_size", "q07_hidden_knowledge_exists"),
    ("q27_decision_level", "q09_hidden_knowledge_value"),
]

original_chi_square_results = pd.DataFrame([
    chi_square_test(df, x, y, analysis_type="original exploratory") for x, y in TEST_PAIRS
])
original_chi_square_results.to_csv(OUTPUT_DIR / "exploratory_chi_square_original.csv", index=False)
original_chi_square_results.to_csv(OUTPUT_DIR / "inferential_statistics.csv", index=False)
original_chi_square_results
```

Out[8]:

	analysis_type	variable_x	variable_y	n	chi2	dof	p_value	bias_corrected_cramers_v	minimum_expected_count	expected_cells_below_5_percent	interpretation
0	original exploratory	q13_personality_similarity	q16_personality_test_willingness	187	7.6609	4	0.104818	0.0992	0.1765	44.44	exploratory due to sparse expected counts
1	original exploratory	q13_personality_similarity	q17_profile_use_consent	187	1.4415	4	0.836949	0.0000	0.3850	33.33	exploratory due to sparse expected counts
2	original exploratory	q18_ai_helpfulness	q19_ai_internal_channel_analysis	187	24.5200	6	0.000419	0.2235	0.2246	33.33	exploratory due to sparse expected counts
3	original exploratory	q18_ai_helpfulness	q21_ai_work_content_analysis	187	8.8435	6	0.182572	0.0872	0.1925	33.33	exploratory due to sparse expected counts
4	original exploratory	q24_organization_size	q07_hidden_knowledge_exists	187	4.9057	2	0.086050	0.1248	2.1551	33.33	exploratory due to sparse expected counts
5	original exploratory	q27_decision_level	q09_hidden_knowledge_value	186	10.0954	8	0.258394	0.0747	0.8602	40.00	exploratory due to sparse expected counts

In [9]:

```python
# Run theoretically collapsed chi-square tests to reduce sparse cells while preserving substantive contrasts.
def collapse_for_chi_square(dataframe):
    collapsed = dataframe.copy()
    collapsed["q13_similarity_collapsed"] = collapsed["q13_personality_similarity"].replace({
        "Yes": "Explicit similarity preference",
        "No": "No explicit similarity preference",
        "It doesn't matter to me": "No explicit similarity preference",
    })
    for source, target in [
        ("q16_personality_test_willingness", "q16_test_willingness_collapsed"),
        ("q17_profile_use_consent", "q17_profile_consent_collapsed"),
        ("q19_ai_internal_channel_analysis", "q19_internal_ai_consent_collapsed"),
        ("q21_ai_work_content_analysis", "q21_work_content_ai_consent_collapsed"),
    ]:
        collapsed[target] = collapsed[source].replace({
            "Yes": "Yes or conditional yes",
            "With certain conditions Yes": "Yes or conditional yes",
            "Up to a certain level Yes": "Yes or conditional yes",
            "No": "No",
        })
    collapsed["q18_ai_helpfulness_collapsed"] = collapsed["q18_ai_helpfulness"].replace({
        "Yes": "Helpful or partly helpful",
        "Up to a certain level Yes": "Helpful or partly helpful",
        "No": "Not helpful",
    })
    collapsed["q24_organization_size_collapsed"] = collapsed["q24_organization_size"].replace({
        "Less than 100": "Less than 500",
        "100-500": "Less than 500",
        "More than 500": "More than 500",
    })
    collapsed["q27_decision_level_collapsed"] = collapsed["q27_decision_level"].replace({
        "Low": "Low",
        "Medium": "Medium or high",
        "High": "Medium or high",
    })
    collapsed["q09_hidden_knowledge_value_collapsed"] = collapsed["q09_hidden_knowledge_value"].replace({
        "Very low": "Low value",
        "Low": "Low value",
        "Medium": "Medium value",
        "Large": "High value",
        "Very large": "High value",
    })
    return collapsed

collapsed_df = collapse_for_chi_square(df)
COLLAPSED_TEST_PAIRS = [
    ("q13_similarity_collapsed", "q16_test_willingness_collapsed"),
    ("q13_similarity_collapsed", "q17_profile_consent_collapsed"),
    ("q18_ai_helpfulness_collapsed", "q19_internal_ai_consent_collapsed"),
    ("q18_ai_helpfulness_collapsed", "q21_work_content_ai_consent_collapsed"),
    ("q24_organization_size_collapsed", "q07_hidden_knowledge_exists"),
    ("q27_decision_level_collapsed", "q09_hidden_knowledge_value_collapsed"),
]

collapsed_chi_square_results = pd.DataFrame([
    chi_square_test(collapsed_df, x, y, analysis_type="collapsed exploratory") for x, y in COLLAPSED_TEST_PAIRS
])
collapsed_chi_square_results.to_csv(OUTPUT_DIR / "chi_square_collapsed_categories.csv", index=False)
collapsed_chi_square_results
```

Out[9]:

	analysis_type	variable_x	variable_y	n	chi2	dof	p_value	bias_corrected_cramers_v	minimum_expected_count	expected_cells_below_5_percent	interpretation
0	collapsed exploratory	q13_similarity_collapsed	q16_test_willingness_collapsed	187	0.4870	1	0.485278	0.0000	3.0000	25.00	exploratory due to sparse expected counts
1	collapsed exploratory	q13_similarity_collapsed	q17_profile_consent_collapsed	187	0.0717	1	0.788874	0.0000	6.5455	0.00	meets expected-count screening criterion
2	collapsed exploratory	q18_ai_helpfulness_collapsed	q19_internal_ai_consent_collapsed	187	1.0998	1	0.294309	0.0225	0.4064	50.00	exploratory due to sparse expected counts
3	collapsed exploratory	q18_ai_helpfulness_collapsed	q21_work_content_ai_consent_collapsed	187	0.2153	1	0.642629	0.0000	0.1925	50.00	exploratory due to sparse expected counts
4	collapsed exploratory	q24_organization_size_collapsed	q07_hidden_knowledge_exists	187	0.2649	1	0.606768	0.0000	4.8663	25.00	exploratory due to sparse expected counts
5	collapsed exploratory	q27_decision_level_collapsed	q09_hidden_knowledge_value_collapsed	186	5.6206	2	0.060186	0.1397	3.2151	16.67	exploratory due to sparse expected counts

In [10]:

```python
# Export contingency tables for the original exploratory chi-square tests.
for variable_x, variable_y in TEST_PAIRS:
    contingency = pd.crosstab(df[variable_x], df[variable_y], dropna=False)
    contingency.to_csv(TABLE_DIR / f"contingency_{variable_x}__{variable_y}.csv")
```

In [11]:

```python
# Apply ordinal coding and calculate Spearman rho and Kendall tau-b for ordered survey variables.
ORDINAL_CODE_MAPS = {
    "q08_knowledge_utilization": {
        "Completely unutilized": 0,
        "Less than 25%": 1,
        "25% to 50%": 2,
        "50% to 75%": 3,
        "More than 75%": 4,
        "100% utilized": 5,
    },
    "q09_hidden_knowledge_value": {"Very low": 0, "Low": 1, "Medium": 2, "Large": 3, "Very large": 4},
    "q10_fear_of_sharing": {"No": 0, "Mostly No": 1, "Mostly Yes": 2, "Yes": 3},
    "q16_personality_test_willingness": {"No": 0, "With certain conditions Yes": 1, "Yes": 2},
    "q17_profile_use_consent": {"No": 0, "With certain conditions Yes": 1, "Yes": 2},
    "q18_ai_helpfulness": {"No": 0, "Up to a certain level Yes": 1, "Yes": 2},
    "q19_ai_internal_channel_analysis": {"No": 0, "Up to a certain level Yes": 1, "With certain conditions Yes": 1, "Yes": 2},
    "q20_ai_public_profile_analysis": {"No": 0, "Up to a certain level Yes": 1, "With certain conditions Yes": 1, "Yes": 2},
    "q21_ai_work_content_analysis": {"No": 0, "Up to a certain level Yes": 1, "With certain conditions Yes": 1, "Yes": 2},
    "q23_age_group": {"20-30": 0, "30-40": 1, "40-50": 2, "50-60": 3, "More than 60": 4},
    "q24_organization_size": {"Less than 100": 0, "100-500": 1, "More than 500": 2},
    "q25_tenure": {"Less than 1 year": 0, "1-5 years": 1, "5-10 years": 2, "More than 10 years": 3},
    "q26_education": {"Lower": 0, "Middle": 1, "Upper": 2, "High": 3},
    "q27_decision_level": {"Low": 0, "Medium": 1, "High": 2},
}

def ordinal_score(series, code_map):
    normalized_map = {normalize_text(k): v for k, v in code_map.items()}
    return series.map(lambda value: normalized_map.get(normalize_text(value), np.nan) if not pd.isna(value) else np.nan)

ordinal_df = df.copy()
for variable, code_map in ORDINAL_CODE_MAPS.items():
    ordinal_df[f"{variable}_ordinal"] = ordinal_score(df[variable], code_map)

ORDINAL_PAIRS = [
    ("q08_knowledge_utilization", "q09_hidden_knowledge_value"),
    ("q10_fear_of_sharing", "q16_personality_test_willingness"),
    ("q16_personality_test_willingness", "q17_profile_use_consent"),
    ("q18_ai_helpfulness", "q19_ai_internal_channel_analysis"),
    ("q18_ai_helpfulness", "q20_ai_public_profile_analysis"),
    ("q18_ai_helpfulness", "q21_ai_work_content_analysis"),
    ("q23_age_group", "q18_ai_helpfulness"),
    ("q24_organization_size", "q08_knowledge_utilization"),
    ("q25_tenure", "q18_ai_helpfulness"),
    ("q26_education", "q18_ai_helpfulness"),
    ("q27_decision_level", "q09_hidden_knowledge_value"),
]

def ordinal_association(dataframe, variable_x, variable_y):
    x = f"{variable_x}_ordinal"
    y = f"{variable_y}_ordinal"
    subset = dataframe[[x, y]].dropna()
    if len(subset) < 3 or subset[x].nunique() < 2 or subset[y].nunique() < 2:
        return {
            "variable_x": variable_x,
            "variable_y": variable_y,
            "n": len(subset),
            "spearman_rho": np.nan,
            "spearman_p_value": np.nan,
            "kendall_tau_b": np.nan,
            "kendall_p_value": np.nan,
            "interpretation": "insufficient ordinal variation",
        }
    rho, rho_p = spearmanr(subset[x], subset[y])
    tau, tau_p = kendalltau(subset[x], subset[y])
    return {
        "variable_x": variable_x,
        "variable_y": variable_y,
        "n": len(subset),
        "spearman_rho": round(float(rho), 4),
        "spearman_p_value": round(float(rho_p), 6),
        "kendall_tau_b": round(float(tau), 4),
        "kendall_p_value": round(float(tau_p), 6),
        "interpretation": "exploratory ordinal association; no causal inference",
    }

ordinal_association_results = pd.DataFrame([ordinal_association(ordinal_df, x, y) for x, y in ORDINAL_PAIRS])
ordinal_association_results.to_csv(OUTPUT_DIR / "ordinal_association_results.csv", index=False)
ordinal_association_results
```

Out[11]:

	variable_x	variable_y	n	spearman_rho	spearman_p_value	kendall_tau_b	kendall_p_value	interpretation
0	q08_knowledge_utilization	q09_hidden_knowledge_value	186	-0.1940	0.007960	-0.1716	0.006507	exploratory ordinal association; no causal inf...
1	q10_fear_of_sharing	q16_personality_test_willingness	187	0.0167	0.820672	0.0156	0.816356	exploratory ordinal association; no causal inf...
2	q16_personality_test_willingness	q17_profile_use_consent	187	0.4754	0.000000	0.4565	0.000000	exploratory ordinal association; no causal inf...
3	q18_ai_helpfulness	q19_ai_internal_channel_analysis	187	0.3106	0.000015	0.2938	0.000022	exploratory ordinal association; no causal inf...
4	q18_ai_helpfulness	q20_ai_public_profile_analysis	187	0.1632	0.025658	0.1556	0.025525	exploratory ordinal association; no causal inf...
5	q18_ai_helpfulness	q21_ai_work_content_analysis	187	0.1998	0.006120	0.1918	0.006488	exploratory ordinal association; no causal inf...
6	q23_age_group	q18_ai_helpfulness	185	0.0257	0.728227	0.0230	0.736117	exploratory ordinal association; no causal inf...
7	q24_organization_size	q08_knowledge_utilization	187	-0.0631	0.391205	-0.0557	0.385767	exploratory ordinal association; no causal inf...
8	q25_tenure	q18_ai_helpfulness	187	0.0737	0.316225	0.0680	0.318016	exploratory ordinal association; no causal inf...
9	q26_education	q18_ai_helpfulness	187	0.0568	0.440250	0.0553	0.437958	exploratory ordinal association; no causal inf...
10	q27_decision_level	q09_hidden_knowledge_value	186	0.0707	0.337301	0.0640	0.326773	exploratory ordinal association; no causal inf...

In [12]:

```python
# Calculate exploratory composite indicators from prepared variables; these are not validated scales.
def score_series(series, score_map):
    normalized_map = {normalize_text(k): v for k, v in score_map.items()}
    return series.map(lambda value: normalized_map.get(normalize_text(value), np.nan) if not pd.isna(value) else np.nan)

def row_mean(dataframe, columns):
    return dataframe[columns].mean(axis=1, skipna=True)

SCORE_MAPS = {
    "yes_no": {"No": 0, "Yes": 1},
    "yes_conditional_no": {"No": 0, "With certain conditions Yes": 0.5, "Up to a certain level Yes": 0.5, "Yes": 1},
    "share": {"No": 0, "With certain compensation Yes": 0.5, "Yes": 1},
    "utilization_inverse": {
        "100% utilized": 0,
        "More than 75%": 0.2,
        "50% to 75%": 0.4,
        "25% to 50%": 0.6,
        "Less than 25%": 0.8,
        "Completely unutilized": 1,
    },
    "value": {"Very low": 0, "Low": 0.25, "Medium": 0.5, "Large": 0.75, "Very large": 1},
    "fear": {"No": 0, "Mostly No": 0.33, "Mostly Yes": 0.67, "Yes": 1},
    "similarity": {"No": 0, "It doesn't matter to me": 0.5, "Yes": 1},
}

analysis_df = df.copy()
analysis_df["exploratory_hidden_knowledge_awareness_composite"] = row_mean(pd.DataFrame({
    "exists": score_series(df["q07_hidden_knowledge_exists"], SCORE_MAPS["yes_no"]),
    "underutilization": score_series(df["q08_knowledge_utilization"], SCORE_MAPS["utilization_inverse"]),
    "economic_value": score_series(df["q09_hidden_knowledge_value"], SCORE_MAPS["value"]),
}), ["exists", "underutilization", "economic_value"])

analysis_df["exploratory_knowledge_sharing_willingness_composite"] = row_mean(pd.DataFrame({
    "colleagues": score_series(df["q11_share_with_colleagues"], SCORE_MAPS["share"]),
    "management": score_series(df["q12_share_with_management"], SCORE_MAPS["share"]),
    "fear_inverse": 1 - score_series(df["q10_fear_of_sharing"], SCORE_MAPS["fear"]),
}), ["colleagues", "management", "fear_inverse"])

analysis_df["exploratory_personality_process_acceptance_composite"] = row_mean(pd.DataFrame({
    "similarity_preference": score_series(df["q13_personality_similarity"], SCORE_MAPS["similarity"]),
    "test_willingness": score_series(df["q16_personality_test_willingness"], SCORE_MAPS["yes_conditional_no"]),
    "profile_consent": score_series(df["q17_profile_use_consent"], SCORE_MAPS["yes_conditional_no"]),
}), ["similarity_preference", "test_willingness", "profile_consent"])

analysis_df["exploratory_ai_readiness_composite"] = row_mean(pd.DataFrame({
    "ai_helpfulness": score_series(df["q18_ai_helpfulness"], SCORE_MAPS["yes_conditional_no"]),
    "internal_channels": score_series(df["q19_ai_internal_channel_analysis"], SCORE_MAPS["yes_conditional_no"]),
    "public_profiles": score_series(df["q20_ai_public_profile_analysis"], SCORE_MAPS["yes_conditional_no"]),
    "work_content": score_series(df["q21_ai_work_content_analysis"], SCORE_MAPS["yes_conditional_no"]),
}), ["ai_helpfulness", "internal_channels", "public_profiles", "work_content"])

analysis_df["exploratory_privacy_sensitivity_composite"] = row_mean(pd.DataFrame({
    "profile_reluctance": 1 - score_series(df["q17_profile_use_consent"], SCORE_MAPS["yes_conditional_no"]),
    "internal_channel_reluctance": 1 - score_series(df["q19_ai_internal_channel_analysis"], SCORE_MAPS["yes_conditional_no"]),
    "public_profile_reluctance": 1 - score_series(df["q20_ai_public_profile_analysis"], SCORE_MAPS["yes_conditional_no"]),
    "work_content_reluctance": 1 - score_series(df["q21_ai_work_content_analysis"], SCORE_MAPS["yes_conditional_no"]),
}), ["profile_reluctance", "internal_channel_reluctance", "public_profile_reluctance", "work_content_reluctance"])

COMPOSITE_COLUMNS = [column for column in analysis_df.columns if column.startswith("exploratory_")]
composite_summary = analysis_df[COMPOSITE_COLUMNS].describe().T.round(3)
composite_summary.to_csv(TABLE_DIR / "exploratory_composite_indicator_summary.csv")
analysis_df.to_csv(OUTPUT_DIR / "data_with_exploratory_composites.csv", index=False)
composite_summary
```

Out[12]:

	count	mean	std	min	25%	50%	75%	max
exploratory_hidden_knowledge_awareness_composite	187.0	0.724	0.155	0.150	0.692	0.767	0.850	1.0
exploratory_knowledge_sharing_willingness_composite	187.0	0.846	0.147	0.223	0.777	0.890	1.000	1.0
exploratory_personality_process_acceptance_composite	187.0	0.798	0.204	0.000	0.667	0.833	1.000	1.0
exploratory_ai_readiness_composite	187.0	0.582	0.244	0.125	0.375	0.500	0.750	1.0
exploratory_privacy_sensitivity_composite	187.0	0.444	0.267	0.000	0.250	0.500	0.625	1.0

In [13]:

```python
# Export an inventory of generated appendix analysis artifacts.
output_inventory = pd.DataFrame({
    "output": [
        "Prepared input dataset",
        "Variable dictionary",
        "Prepared dataset schema audit",
        "Missing-value summary",
        "Unique-values audit",
        "Combined frequency tables",
        "Combined percentage tables",
        "Corrected q03 multi-select frequency table",
        "Original exploratory chi-square table",
        "Collapsed-category chi-square table",
        "Ordinal association table",
        "Exploratory composite indicator summary",
        "Data with exploratory composite indicators",
        "Plots directory",
    ],
    "path": [
        str(DATA_PATH),
        str(VARIABLE_DICTIONARY_PATH),
        str(TABLE_DIR / "prepared_dataset_schema_audit.csv"),
        str(TABLE_DIR / "missing_values.csv"),
        str(TABLE_DIR / "unique_values_by_column.csv"),
        str(TABLE_DIR / "all_frequency_tables.csv"),
        str(TABLE_DIR / "all_percentage_tables.csv"),
        str(TABLE_DIR / "q03_solution_source_multiselect_summary.csv"),
        str(OUTPUT_DIR / "exploratory_chi_square_original.csv"),
        str(OUTPUT_DIR / "chi_square_collapsed_categories.csv"),
        str(OUTPUT_DIR / "ordinal_association_results.csv"),
        str(TABLE_DIR / "exploratory_composite_indicator_summary.csv"),
        str(OUTPUT_DIR / "data_with_exploratory_composites.csv"),
        str(PLOT_DIR),
    ],
})
output_inventory.to_csv(OUTPUT_DIR / "output_inventory.csv", index=False)
output_inventory
```

Out[13]:

	output	path
0	Prepared input dataset	data/data_cleaned.csv
1	Variable dictionary	data/variable_dictionary.csv
2	Prepared dataset schema audit	outputs/descriptive_tables/prepared_dataset_sc...
3	Missing-value summary	outputs/descriptive_tables/missing_values.csv
4	Unique-values audit	outputs/descriptive_tables/unique_values_by_co...
5	Combined frequency tables	outputs/descriptive_tables/all_frequency_table...
6	Combined percentage tables	outputs/descriptive_tables/all_percentage_tabl...
7	Corrected q03 multi-select frequency table	outputs/descriptive_tables/q03_solution_source...
8	Original exploratory chi-square table	outputs/exploratory_chi_square_original.csv
9	Collapsed-category chi-square table	outputs/chi_square_collapsed_categories.csv
10	Ordinal association table	outputs/ordinal_association_results.csv
11	Exploratory composite indicator summary	outputs/descriptive_tables/exploratory_composi...
12	Data with exploratory composite indicators	outputs/data_with_exploratory_composites.csv
13	Plots directory	outputs/plots